

Government Job Tracker & Alert System

Special PEP Project

A training report

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

(Computer Science & Engineering)

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB



From 06/20/2026 to 08/07/2026

SUBMITTED BY

Name of student: Kriti Pandey

Registration Number: 12407789

Signature of the student:

Declaration

To whomsoever it may concern,

I, **Kriti Pandey, 12407789**, hereby declare that the work presented in this project report titled **“Government Job Tracker & Alert System”** is a record of original work carried out by me in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (Computer Science & Engineering) under the Special PEP framework at Lovely Professional University, and that it has not been submitted elsewhere for the award of any other degree or diploma.

Signature of the student -

Kriti Pandey

Date: 20th August 2026

Training Certificate



Acknowledgement

I would like to express my sincere gratitude to my mentor, Mam Prerita, and the Department of Computer Science & Engineering, Lovely Professional University, for their guidance, feedback and support throughout the design, development and testing of this project. I also thank my peers and family for their continuous encouragement during the course of this work.

Abstract

The Government Job Tracker & Alert System is a full-stack application designed to organize government job postings and make deadline-oriented job discovery easier. The project began as a command-line application and was extended into a web-oriented architecture with a FastAPI REST backend and a React/Vite presentation layer. Its primary academic focus is the practical use of Data Structures and Algorithms within a working software system.

The core engine uses a Trie for prefix-based keyword search, a binary Min-Heap for deadline-priority alerts, a hand-written Merge Sort for ordering postings by deadline, Binary Search for deadline range queries, and hash-based structures for constant-time average-case lookup and duplicate detection. The same JobTracker engine is exposed through the CLI and REST API, allowing the DSA implementation to remain independent from the presentation layer.

The current version uses representative mock government-job data and in-memory application state. Automated unit and API integration tests cover the core data structures, tracker behaviour, and REST endpoints. The system is therefore suitable as a Special PEP academic project demonstrating how classical DSA concepts can be integrated into a modern web application while retaining a clear separation between the algorithmic engine and the user interface.

Table of Contents

Declaration.....	2
Training Certificate.....	3
Acknowledgement	4
Abstract.....	5
List of Figures.....	11
List of Abbreviations	12
Chapter-1 Introduction of the Project Undertaken	13
1.1 Objectives of the Work Undertaken	13
1.2 Scope of the Work	13
1.3 Importance and Applicability	14
1.4 Role and Profile	14
1.5 Work Plan and Implementation Approach	14
Chapter-2 System Architecture and Design	15
2.1 Architectural Overview	15
2.2 Technology Stack	15
2.3 Application Startup and State Management	16
2.4 Cross-Origin Resource Sharing	16
Chapter-3 Data Structures and Algorithms - Core Implementation	17
3.1 Job Data Model and Hash-Based Duplicate Detection.....	17
3.2 Trie (Prefix Tree) - Keyword Search.....	17
3.3 Min-Heap - Deadline Priority Queue	18
3.4 Merge Sort - Stable $O(n \log n)$ Sort by Deadline	18
3.5 Binary Search - Range Queries on Sorted Deadlines	18
3.6 Complexity Summary	19

Chapter-4 Implementation - REST API, CLI and Alerting	20
4.1 REST API - FastAPI Application.....	20
4.2 Request and Response Schemas	21
4.3 Command-Line Interface.....	21
4.4 Alerting - Console and E-mail.....	21
4.5 Sample Dataset	22
Chapter-5 Testing, Results, Limitations and Future Enhancements.....	23
5.1 Testing Strategy	23
5.1.1 Unit Tests - test_dsa.py.....	23
5.1.2 API Integration Tests - test_api.py	23
5.2 Results - Website Screenshots	24
5.3 Limitations.....	27
5.4 Future Enhancements.....	27
Final Chapter - Conclusion.....	28
References.....	29
Appendix A: Detailed DSA Walkthroughs - Trie	30
A.1 Character-by-Character Traversal.....	30
A.2 Why the Structure Fits the Requirement.....	30
A.3 Edge Cases	30
A.4 Complexity.....	30
Appendix B: Detailed DSA Walkthroughs - Min-Heap.....	31
B.1 Heap Invariant.....	31
B.2 Insertion	31
B.3 Removal	31
B.4 Non-Destructive Preview	31

B.5 Complexity.....	32
Appendix C: Detailed DSA Walkthroughs - Merge Sort	33
C.1 Divide Phase	33
C.2 Merge Phase.....	33
C.3 Deadline Ordering.....	33
C.4 Why Implement It Manually	33
C.5 Complexity.....	33
Appendix D: Detailed DSA Walkthroughs - Binary Search	34
D.1 Search Invariant	34
D.2 Lower Bound	34
D.3 Upper Bound.....	34
D.4 Important Qualification.....	34
D.5 Complexity.....	35
Appendix E: Hashing and Duplicate Detection.....	36
E.1 Composite Deduplication Key	36
E.2 Why Hashing Fits.....	36
E.3 Lookup	36
E.4 Limitations	36
Appendix F: JobTracker Integration.....	37
F.1 Central Coordination	37
F.2 Add-Job Flow	37
F.3 Search Flow	37
F.4 Alert Flow.....	37
F.5 Separation of Concerns.....	37
Appendix G: FastAPI Endpoint Reference.....	38

G.1 Endpoint Design	38
G.2 Example Request	38
G.3 Duplicate Response.....	38
G.4 API Documentation	39
Appendix H: Frontend and User Workflow	40
H.1 Dashboard Workflow.....	40
H.2 Explore Jobs Workflow	40
H.3 Deadline Alerts Workflow.....	40
H.4 Post Job Workflow.....	40
Appendix I: Testing Matrix	41
I.1 Automated Test Matrix	41
I.2 Test Quality	41
I.3 Regression Value.....	41
Appendix J: Manual Test Scenarios	42
J.1 Search Scenario	42
J.2 Deadline Scenario	42
J.3 Duplicate Scenario	42
J.4 Details Scenario	42
Appendix K: Performance and Complexity Discussion.....	43
K.1 Asymptotic View	43
K.2 Current Dataset	43
K.3 Future Measurement	43
Appendix L: Security and Reliability Considerations	44
L.1 Credential Handling	44
L.2 Input Validation.....	44

L.3 Deployment Considerations	44
Appendix M: Feasibility and Deployment Roadmap	45
M.1 Technical Feasibility	45
M.2 Economic Feasibility.....	45
M.3 Operational Feasibility	45
M.4 Roadmap	45
Appendix N: User Guide	46
N.1 CLI Demonstration	46
N.2 Backend	46
N.3 Frontend	46
N.4 Demonstration Order	46
Appendix O: Academic Learning Outcomes.....	47
O.1 DSA Learning	47
O.2 Software Engineering Learning	47
O.3 Problem-Solving Learning.....	47
O.4 Evolution of the Project	47
Appendix P: Future Enhancement Design Notes	48
P.1 Persistent Storage	48
P.2 Live Sources	48
P.3 Personalization	48
P.4 Advanced DSA.....	48

List of Figures

Figure 2.1: Layered architecture of the Government Job Tracker & Alert System15

Figure 5.1: Website dashboard24

Figure 5.2: Explore Jobs interface25

Figure 5.3: Deadline Alerts interface25

Figure 5.4: Post Job interface26

List of Abbreviations

Table 1: Abbreviations and their full forms

Abbreviation	Full Form
API	Application Programming Interface
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
DSA	Data Structures and Algorithms
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
UI	User Interface

Chapter-1 Introduction of the Project Undertaken

1.1 Objectives of the Work Undertaken

- To design and implement a full-stack application, the “Government Job Tracker & Alert System”, that tracks Indian government job postings and generates deadline-based alerts for applicants.
- To apply core Data Structures and Algorithms - a Trie (prefix tree), a Min-Heap (priority queue), a hand-written Merge Sort, Binary Search range queries, and hash-based duplicate detection - as the functional engine of a working system rather than as isolated textbook exercises.
- To expose the tracker’s functionality through a documented, testable REST API built with FastAPI, so that the backend can be consumed independently by a web front end or another client.
- To validate the correctness of the data-structure implementations and API layer through automated unit and integration tests.

1.2 Scope of the Work

The scope of the present work covers the Job data model, JobTracker orchestration layer, individual DSA modules, REST API layer built on FastAPI with Pydantic-validated schemas, command-line interface, console/e-mail alerting utility, and automated tests for the DSA and API layers.

The current application uses representative mock Indian government job postings. The data source covers categories such as Staff Selection Commission, banking/IBPS, Railways/RRB, UPSC, State Police, State Public Service Commissions, Defence, State PSUs, Central Government departments, Insurance and Education. Posting and deadline dates are generated relative to the current date so that the demonstration contains a useful mixture of expired, closing-soon and future jobs.

Live scraping of government job portals, persistent database storage, user authentication, scheduled background alerting and production deployment are outside the implemented scope of the current version. They are treated as future enhancements rather than completed features.

1.3 Importance and Applicability

Government recruitment information in India is distributed across many recruiting bodies and portals. A candidate preparing for multiple examinations may therefore need to track different notifications and application deadlines. A searchable and deadline-aware tracker provides a practical context in which DSA techniques can be applied to a recognizable software problem.

The project also demonstrates the importance of selecting an appropriate data structure for an operation. A Trie supports prefix search without repeatedly scanning every record, a Min-Heap provides priority access to the nearest deadline, Merge Sort provides predictable $O(n \log n)$ ordering, Binary Search supports logarithmic boundary searches on sorted deadlines, and hashing provides average $O(1)$ lookup and duplicate detection.

1.4 Role and Profile

This was an individual Special PEP academic project. The student was responsible for requirement analysis, DSA selection and implementation, JobTracker integration, CLI development, REST API development, alerting, automated testing, web-interface integration and preparation of the project documentation.

1.5 Work Plan and Implementation Approach

The implementation followed a bottom-up approach. Individual data structures were implemented and tested first. The Job model and mock data source were then created, followed by JobTracker as the coordinating layer. The CLI provided the first end-to-end interface. The same tracker was subsequently exposed through FastAPI, enabling a web client to communicate with the DSA engine through HTTP and JSON.

Chapter-2 System Architecture and Design

2.1 Architectural Overview

The system follows a layered architecture. The presentation layer communicates with a FastAPI REST backend, while the backend delegates application operations to the JobTracker engine. JobTracker owns the core indexes and DSA structures. The CLI remains an alternate presentation path that can exercise the same engine without HTTP.

This separation is important for the project because changing the interface from CLI to web does not require rewriting the DSA implementations. The Trie, Min-Heap, Merge Sort, Binary Search and hash-based structures remain concentrated in the core engine.

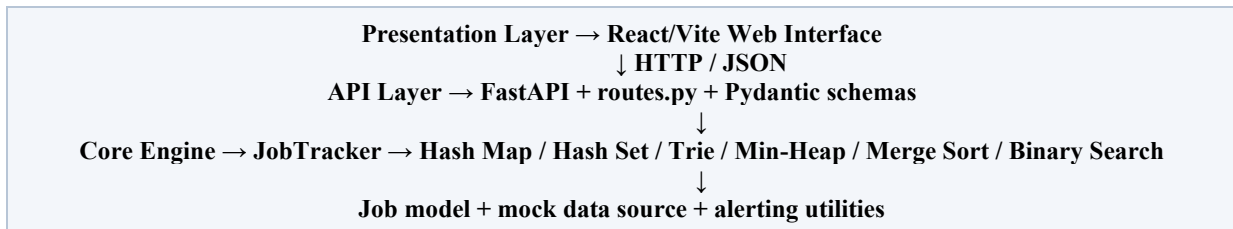


Figure 2.1: Layered architecture of the Government Job Tracker & Alert System

2.2 Technology Stack

Table 2.1: Technology stack used in the project

Layer	Technology
Backend language	Python
Web framework	FastAPI
Data validation	Pydantic
ASGI server	Uvicorn
Testing	Python unittest and FastAPI TestClient
Email alerting	smtplib with Gmail SMTP over SSL
Front end	React with Vite
API documentation	Swagger UI and ReDoc through FastAPI

2.3 Application Startup and State Management

On startup, the FastAPI application creates a JobTracker instance, loads the sample dataset and stores the tracker in application state. Request handlers retrieve this shared tracker instance. Consequently, jobs, the Trie index and the deadline heap remain available across requests during the lifetime of the server process.

Because the current application is in-memory, restarting the server resets the state and reloads the sample dataset. This is a deliberate limitation of the current academic version and provides a clear future extension point for persistent storage.

2.4 Cross-Origin Resource Sharing

The backend includes CORS middleware so that a separately served web client can communicate with the API during development. The current development-oriented configuration is permissive. A production deployment should restrict allowed origins to the actual front-end domain.

Chapter-3 Data Structures and Algorithms - Core Implementation

3.1 Job Data Model and Hash-Based Duplicate Detection

The Job class is a dataclass representing one government job posting. It contains identifiers, descriptive fields, dates, an application link and the number of vacancies.

Table 3.1: Job data model - fields and types

Field	Type	Purpose
job_id	str	Unique job identifier
title / department / category	str	Descriptive fields indexed for keyword search
location / qualification	str	Descriptive eligibility and location information
post_date / deadline	date	Used for ordering, range queries and alerts
apply_link	str	Application portal URL
vacancies	int	Number of available positions

A normalized deduplication key is built from the job title, department and post date. A Python hash set stores previously seen keys. Membership testing therefore has average $O(1)$ time complexity, allowing a new posting to be rejected before it is inserted into the indexes.

3.2 Trie (Prefix Tree) - Keyword Search

The Trie implementation stores characters in a tree of nodes. Each node maintains a set of job identifiers associated with the prefix represented by that node. This design means that a prefix can be resolved by following its characters without traversing all matching records.

- `insert(word, job_id)` lower-cases the word and creates or follows one node for each character.
- `insert_text(text, job_id)` tokenizes longer text and indexes individual words.
- `search_prefix(prefix)` walks the Trie according to the query characters and returns the associated job identifiers.

- JobTracker indexes title, department, category and location, allowing a prefix such as “bank” to match relevant postings.

For a query of length L , Trie traversal is $O(L)$, excluding the cost of processing the returned result set.

3.3 Min-Heap - Deadline Priority Queue

The deadline alert mechanism uses a binary Min-Heap stored in an array. Each heap item contains a deadline priority and job identifier. The smallest deadline remains at the root, making the most urgent posting immediately accessible.

- push performs insertion followed by sift-up and takes $O(\log n)$.
- pop_min removes the root, restores heap order using sift-down and takes $O(\log n)$.
- peek returns the root without removal and takes $O(1)$.
- peek_all_due_alerts uses a temporary copy so that API polling does not consume the real alert heap.
- pop_all_due_alerts destructively consumes due alerts, which is useful for one-time console/e-mail notifications.

3.4 Merge Sort - Stable $O(n \log n)$ Sort by Deadline

The project implements Merge Sort rather than relying on Python’s built-in sort. The list is recursively divided until sublists contain at most one element. The merge operation then combines the sorted sublists while comparing the requested key, such as deadline.

The resulting complexity is $O(n \log n)$ in the usual best, average and worst cases. The implementation can therefore demonstrate the mechanics and complexity of a classical comparison-based sorting algorithm directly within the project.

3.5 Binary Search - Range Queries on Sorted Deadlines

Binary Search is used after the jobs have been ordered by deadline. Lower-bound and upper-bound operations locate the first relevant positions for a date interval. The resulting slice contains jobs whose deadlines fall within the requested range.

The binary-search portion of the operation is $O(\log n)$. In the current implementation, the surrounding operation first constructs the merge-sorted view, so the complete closing-within query also includes the sorting cost when a fresh sorted list is required.

3.6 Complexity Summary

Table 3.2: Complexity summary of the core data structures and algorithms

Structure / Algorithm	Operation	Complexity	Use
Hash Map	Job lookup by ID	$O(1)$ average	jobs_by_id
Hash Set	Duplicate detection	$O(1)$ average	seen-key index
Trie	Prefix search	$O(L)$	keyword search
Min-Heap	Push / pop	$O(\log n)$	deadline alerts
Min-Heap	Peek	$O(1)$	alert preview
Merge Sort	Deadline sorting	$O(n \log n)$	sorted listings
Binary Search	Deadline bounds	$O(\log n)$	range query

Chapter-4 Implementation - REST API, CLI and Alerting

4.1 REST API - FastAPI Application

The FastAPI application provides an HTTP interface to the JobTracker engine. It exposes interactive API documentation through Swagger UI and ReDoc, registers the API routes and initializes the tracker with the sample dataset.

Table 4.1: REST API endpoints exposed by the FastAPI backend

Method	Path	Description	Algorithm / Mechanism
GET	/	Health check	Service status
GET	/api/jobs	List jobs / optional deadline sort	Hash Map / Merge Sort
GET	/api/jobs/search	Prefix search	Trie O(L)
GET	/api/jobs/closing-soon	Jobs closing within N days	Merge Sort + Binary Search
GET	/api/alerts	Preview due alerts	Min-Heap
POST	/api/alerts/email	Send e-mail alert	Min-Heap + SMTP
POST	/api/jobs	Create job	Hash Set + Trie + Min-Heap
GET	/api/stats	Tracker statistics	Counters / heap length

List responses include information identifying the algorithm used and execution timing. This makes the DSA role visible during demonstrations instead of hiding the implementation behind a generic list operation.

4.2 Request and Response Schemas

Table 4.2: Pydantic schemas used for request/response validation

Schema	Purpose
JobCreate	Validates data submitted when creating a job
JobResponse	Serializes a Job and provides computed deadline information
JobListResponse	Wraps job lists with total, algorithm and execution time
EmailAlertRequest / EmailAlertResponse	Handles e-mail alert requests and results
TrackerStatsResponse	Returns tracker counters

Pydantic schemas provide a clear contract between the web client and backend. Invalid date values can be rejected before they reach the core tracker logic.

4.3 Command-Line Interface

The original CLI remains available as an alternate entry point. It provides menu options for keyword search, deadline sorting, closing-soon queries, alerts, e-mail notifications, tracker statistics and exit. A `--demo` mode provides a non-interactive demonstration of the major features.

Keeping the CLI is useful academically because it demonstrates that the DSA engine is independent of the web interface. The same JobTracker methods can therefore be exercised through two different presentation mechanisms.

4.4 Alerting - Console and E-mail

Console alerting requires no external configuration and prints the due postings to the terminal. E-mail alerting uses Python's `smtplib` to communicate with Gmail SMTP over SSL. The sender address and application-specific password are read from environment variables rather than being hard-coded in source code.

The use of environment variables is important because credentials should never be committed to the source repository. In a production system, additional secret management and authentication controls would be required.

4.5 Sample Dataset

The current data source provides representative mock Indian government job postings covering several recruitment categories. Dates are generated relative to the current date, allowing the demonstration to continue showing meaningful deadline states as time passes. The dataset is intended to support stable testing of the DSA engine; live data ingestion is a future enhancement.

Chapter-5 Testing, Results, Limitations and Future Enhancements

5.1 Testing Strategy

The project contains automated tests at two levels. The DSA test suite verifies individual data structures and their integration through JobTracker. The API test suite verifies the REST endpoints using FastAPI's test client, providing integration-level validation of the web backend.

5.1.1 Unit Tests - test_dsa.py

Table 5.1: Unit test coverage

Test Area	Coverage
Trie	Insertion, prefix search, tokenization and invalid/empty prefix handling
Min-Heap	Empty behaviour, ordering, push/pop and heap operations
Merge Sort	Numeric sorting, Job sorting with key functions, edge cases
Binary Search	Lower/upper bounds and closing-between range queries
Duplicate Detection	Dedup-key normalization and duplicate rejection
JobTracker Integration	Search, sorting, range query and alert behaviour

5.1.2 API Integration Tests - test_api.py

Table 5.2: Integration test coverage

Endpoint / Test Area	Verification
Root endpoint	Successful health/status response
GET /api/jobs	Seeded job retrieval
Sorted jobs	Deadline ordering and Merge Sort indication

Endpoint / Test Area	Verification
Trie search	Correct prefix results
Closing soon	Deadline window correctness
Alerts	Repeated non-destructive alert calls
Create job	Successful creation and duplicate 409 response
Stats	Tracker counters

5.2 Results - Website Screenshots

The web interface provides a presentation layer for the same backend functionality.

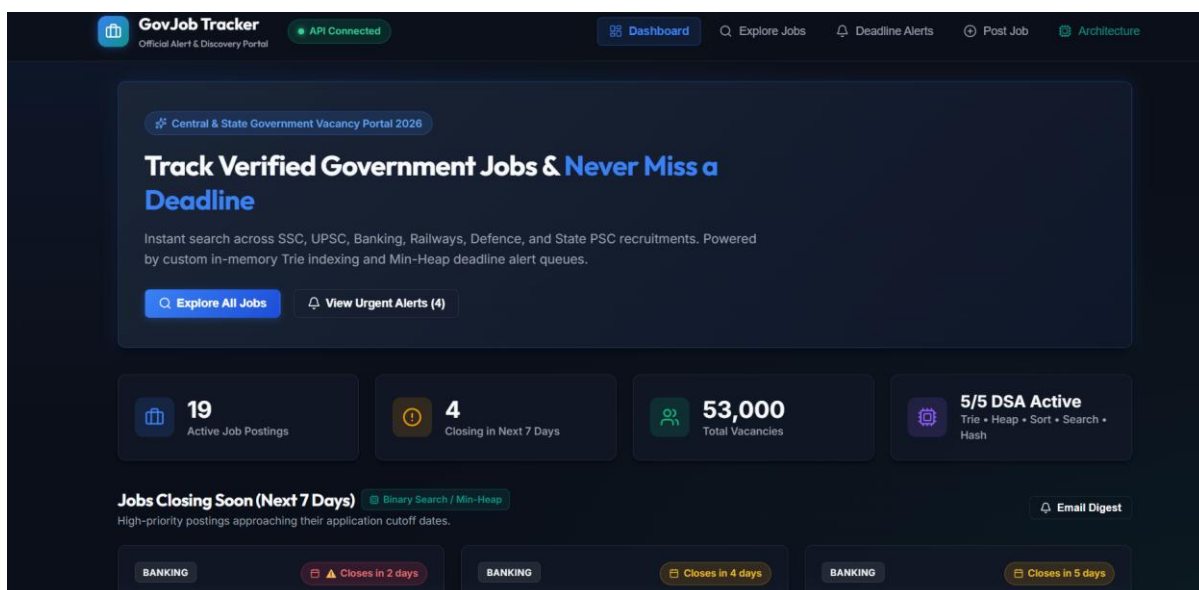


Figure 5.1: Website dashboard

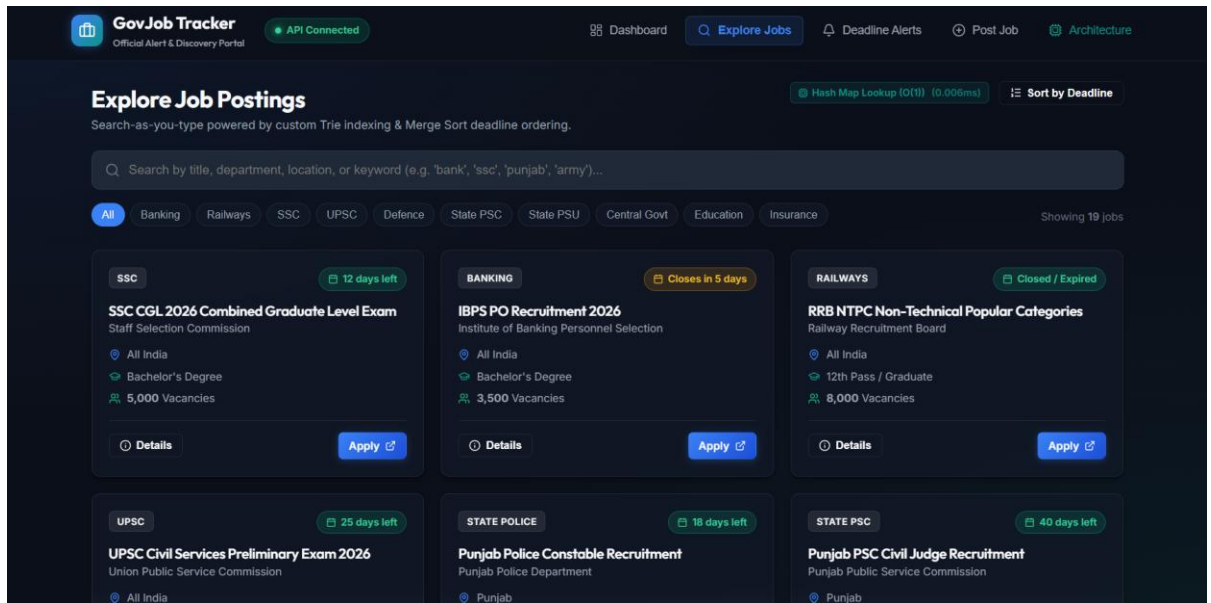


Figure 5.2: Explore Jobs interface

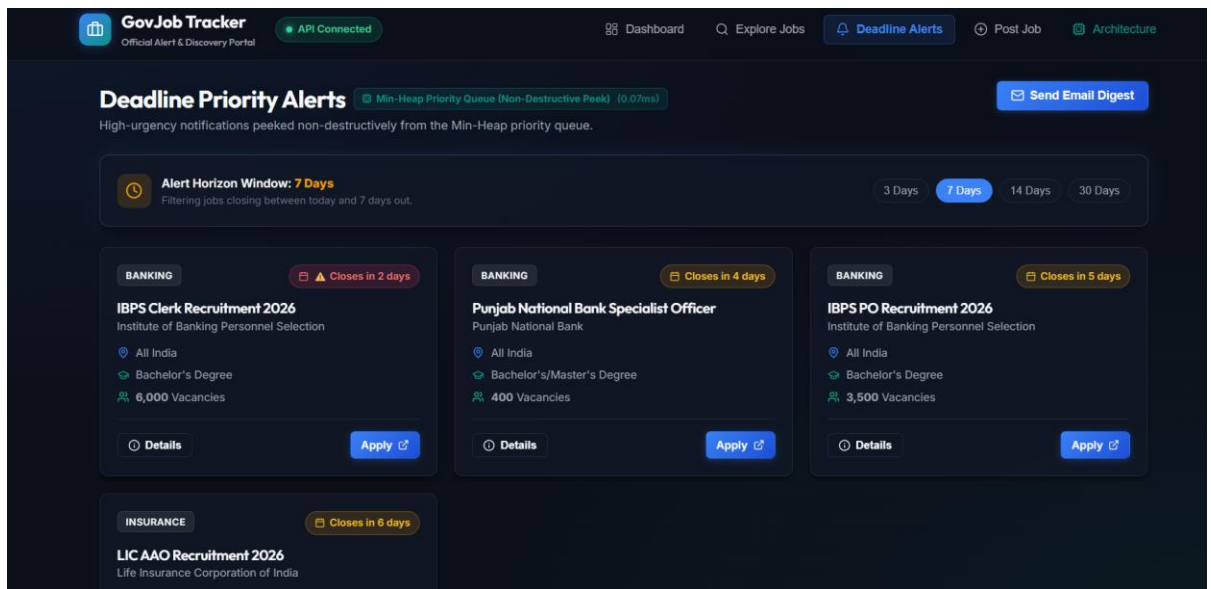
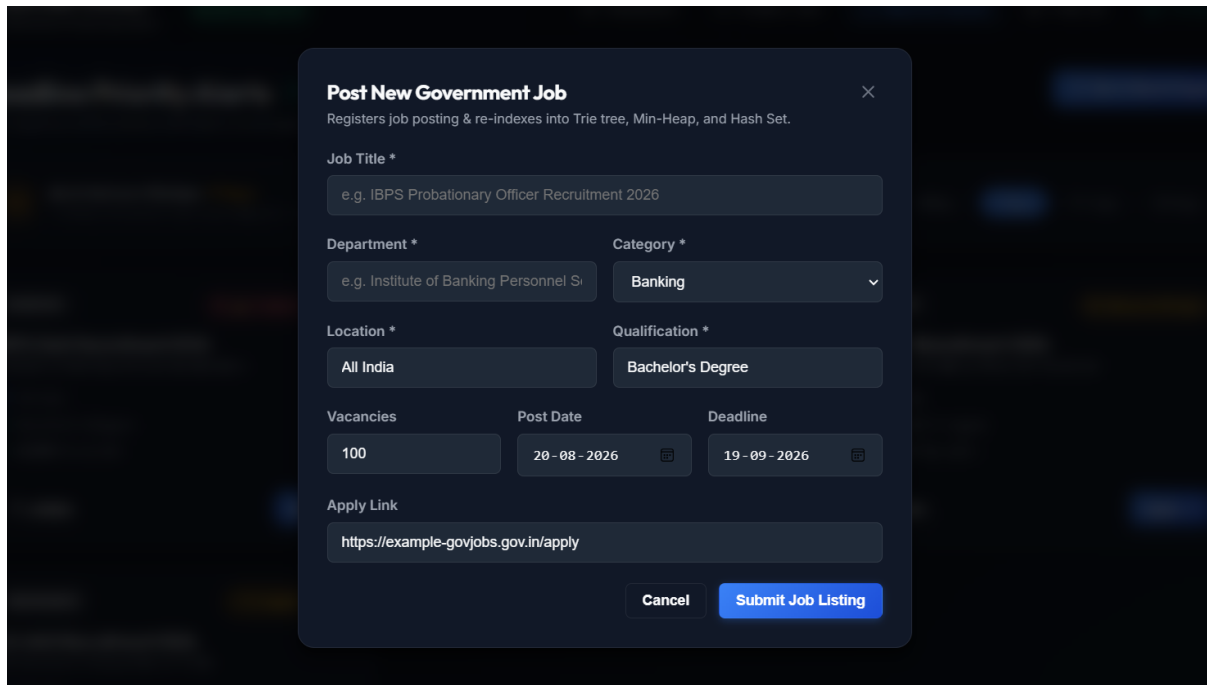


Figure 5.3: Deadline Alerts interface



The image shows a 'Post New Government Job' form with a dark theme. The form is titled 'Post New Government Job' with a close button (X) in the top right. Below the title is a subtitle: 'Registers job posting & re-indexes into Trie tree, Min-Heap, and Hash Set.' The form contains several input fields and buttons:

- Job Title ***: A text input field with the placeholder text 'e.g. IBPS Probationary Officer Recruitment 2026'.
- Department ***: A text input field with the placeholder text 'e.g. Institute of Banking Personnel Selection'.
- Category ***: A dropdown menu with 'Banking' selected and a downward arrow.
- Location ***: A text input field with 'All India' entered.
- Qualification ***: A text input field with 'Bachelor's Degree' entered.
- Vacancies**: A text input field with '100' entered.
- Post Date**: A date input field with '20-08-2026' and a calendar icon.
- Deadline**: A date input field with '19-09-2026' and a calendar icon.
- Apply Link**: A text input field with 'https://example-govjobs.gov.in/apply' entered.
- Buttons**: 'Cancel' and 'Submit Job Listing' buttons at the bottom right.

Figure 5.4: Post Job interface

5.3 Limitations

- In-memory storage only: the current tracker state is reset when the process restarts.
- Mock rather than live data: the current dataset does not fetch postings directly from government portals.
- No authentication or authorization is currently implemented for API access.
- List endpoints do not currently use pagination, which would become important for a large dataset.
- Alerts are on-demand rather than triggered by a background scheduler.
- The current development CORS configuration should be restricted before public deployment.
- Duplicate detection is based on normalized title, department and post date, so semantically identical postings with substantially different wording may not be recognized as duplicates.

5.4 Future Enhancements

The following improvements are intentionally presented as future work rather than current features:

- Persistent storage using SQLite or another suitable database.
- Live data ingestion from selected recruitment sources, subject to their terms and access policies.
- Scheduled background alerting so users do not need to trigger checks manually.
- Pagination and rate limiting for larger datasets and public use.
- User accounts, authentication and personalized alert preferences.
- Additional notification channels such as Telegram or push notifications.
- Improved search capabilities such as fuzzy or typo-tolerant matching.
- Production deployment with restricted CORS, secure secrets and appropriate access controls.

Final Chapter - Conclusion

The Government Job Tracker & Alert System demonstrates how classical Data Structures and Algorithms can serve as the functional core of a practical software application. The project began as a command-line DSA application and was extended into a web-oriented system without replacing the underlying algorithmic engine.

A Trie provides prefix-based keyword search, a binary Min-Heap maintains deadline priority, Merge Sort orders postings by deadline, Binary Search identifies boundaries for deadline ranges, and hashing provides average constant-time lookup and duplicate detection. These structures are coordinated by JobTracker and exposed through both the CLI and FastAPI REST API.

The project also demonstrates software-engineering practices beyond the individual algorithms: modular source organization, API schemas, automated testing, separation between presentation and business logic, and secure handling of e-mail credentials through environment variables.

The present version intentionally uses mock data and in-memory state so that the DSA behaviour remains deterministic and easy to demonstrate. Persistent storage, live data ingestion, scheduled alerts, authentication and production deployment are logical next steps. The architecture allows these additions to be made around the existing DSA engine rather than requiring a complete rewrite.

References

- FastAPI, “FastAPI Documentation,” <https://fastapi.tiangolo.com>.
- Pydantic, “Pydantic Documentation,” <https://docs.pydantic.dev>.
- Python Software Foundation, “Python Documentation,” <https://docs.python.org/3/>.
- Python Software Foundation, “unittest - Unit testing framework,” Python Documentation.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.
- Lovely Professional University, University-provided project report format and guidelines.
- Project source code and README: Government Job Tracker & Alert System.

Appendix A: Detailed DSA Walkthroughs - Trie

This appendix gives a worked explanation of how the Trie is used in the project. The purpose is to make the data-structure implementation easy to demonstrate during evaluation. A Trie stores characters along paths from a root node. Each path represents a prefix, and a node can retain the job identifiers associated with that prefix. This is particularly appropriate for the application's search requirement because users commonly enter the beginning of a department, category or job title rather than the complete phrase.

A.1 Character-by-Character Traversal

Suppose the indexed text contains the token “banking”. When the query “bank” is received, traversal begins at the root and follows $b \rightarrow a \rightarrow n \rightarrow k$. The node reached after k represents the prefix. The identifiers associated with that node identify postings whose indexed text contains that prefix. No comparison with unrelated prefixes is required.

A.2 Why the Structure Fits the Requirement

The requirement is prefix search rather than exact equality. A hash table is excellent for exact-key lookup, but it does not naturally provide all keys beginning with a prefix. A Trie is therefore a better fit for this particular operation. The cost of following the query is proportional to its length, represented as $O(L)$.

A.3 Edge Cases

- Empty prefixes are handled explicitly rather than being treated as a normal search.
- Text is normalized before indexing so that differences in capitalization do not create separate branches for the same word.
- Punctuation and token separators are handled during text indexing.

A.4 Complexity

Operation	Complexity	Interpretation
Insert one token	$O(L)$	One node/path step per character
Prefix search	$O(L) + \text{output}$	Traversal plus returned IDs
Space	$O(\text{total indexed characters})$	Shared prefixes reduce duplication

Appendix B: Detailed DSA Walkthroughs - Min-Heap

The deadline alert queue is modeled as a binary Min-Heap. The priority of an item is its application deadline, so the earliest deadline is placed at the root. This is a direct match between the problem's meaning of urgency and the ordering invariant of the data structure.

B.1 Heap Invariant

For every parent node in the binary Min-Heap, the parent deadline is less than or equal to the deadlines of its children. Because of this invariant, the root always represents the earliest deadline currently stored in the heap.

B.2 Insertion

A new alert is appended at the end of the underlying array. The implementation then repeatedly compares the new item with its parent and swaps upward until the invariant is restored. Each level moves the item at most once, giving $O(\log n)$ insertion.

B.3 Removal

To remove the earliest alert, the root is saved, the final array item is moved to the root, and the implementation performs sift-down. The smaller child is selected when both children exist. The operation stops when the parent is no longer greater than either child.

B.4 Non-Destructive Preview

A web dashboard may need to display alerts repeatedly. Removing items every time the page is refreshed would be incorrect. The project therefore includes a non-destructive preview method that works on a copy, while the destructive pop operation remains available for one-time delivery workflows.

B.5 Complexity

Operation	Complexity
Peek	$O(1)$
Push	$O(\log n)$
Pop minimum	$O(\log n)$
Build from n items	$O(n)$ when heapified appropriately

Appendix C: Detailed DSA Walkthroughs - Merge Sort

The project uses a hand-written Merge Sort to order jobs by deadline. This appendix expands the algorithm into its divide, recursive-sort and merge phases and explains why the implementation is useful in a DSA project even though Python already provides a highly optimized built-in sorting function.

C.1 Divide Phase

Merge Sort divides the input list into two halves until each sublist contains at most one element. A one-element list is already sorted. The recursion therefore creates a set of small sorted lists that can be combined.

C.2 Merge Phase

The merge operation compares the current item from each sorted half and copies the smaller item into the output. Once one half is exhausted, the remaining items of the other half are appended. Repeating this process at every recursion level produces a globally sorted list.

C.3 Deadline Ordering

The project passes the deadline field as the sorting key. The result is a list in chronological order, allowing the UI and CLI to present the most urgent postings first.

C.4 Why Implement It Manually

Python's built-in sort is normally the better engineering choice for a general application. In this academic project, a manual Merge Sort makes the algorithm visible and testable, directly demonstrating divide-and-conquer sorting and $O(n \log n)$ behavior.

C.5 Complexity

Case	Time	Extra Space
Best	$O(n \log n)$	$O(n)$
Average	$O(n \log n)$	$O(n)$
Worst	$O(n \log n)$	$O(n)$

Appendix D: Detailed DSA Walkthroughs - Binary Search

Binary Search is applied to a list that has already been ordered by deadline. Instead of checking every posting, the search repeatedly halves the remaining interval to find the lower and upper boundaries of the requested date range.

D.1 Search Invariant

Binary Search requires an ordered sequence. The project first obtains a deadline-sorted view and then searches for boundary positions. At every step the middle position is compared with the target date, allowing half of the remaining search interval to be discarded.

D.2 Lower Bound

The lower-bound search identifies the first position whose deadline is not earlier than the requested start date. This position becomes the beginning of the result interval.

D.3 Upper Bound

The upper-bound search identifies the first position whose deadline is later than the requested end date. The elements before this boundary and after the lower boundary form the requested range.

D.4 Important Qualification

The binary-search operation itself is $O(\log n)$, but the current closing-within implementation may first create a sorted list. Therefore the end-to-end query is not simply $O(\log n)$ in the current implementation. This distinction is important in an academic complexity discussion because the cost of preparation must be included.

D.5 Complexity

Component	Complexity
Lower bound	$O(\log n)$
Upper bound	$O(\log n)$
Boundary query	$O(\log n + k)$ after sorting
Current fresh sorted view	$O(n \log n)$ preparation

Appendix E: Hashing and Duplicate Detection

Hashing provides the indexing mechanism used to prevent duplicate postings and to support fast job lookup. This appendix explains normalization, key construction, average-case complexity and the practical limitations of using a title/department/post-date composite key.

E.1 Composite Deduplication Key

A posting is normalized into a key containing its title, department and post date. The tracker stores these keys in a set. Before insertion, the key is checked; an existing key causes the posting to be rejected.

E.2 Why Hashing Fits

The system needs to answer a membership question: has this posting already been seen? A hash set is a natural choice because membership is average $O(1)$. This avoids repeatedly scanning every existing job before accepting a new record.

E.3 Lookup

The tracker also maintains a mapping from job identifier to Job object. This is an average $O(1)$ hash-map lookup and is useful when an API endpoint needs to retrieve a specific record.

E.4 Limitations

Hashing does not guarantee $O(1)$ in every theoretical case because collisions can occur. In normal Python dictionary/set use, average-case constant-time membership is the appropriate complexity statement. The deduplication rule is also only as strong as its key fields.

Appendix F: JobTracker Integration

JobTracker is the coordinating layer that turns the individual DSA modules into one application. The important design principle is that the caller should not need to know which data structure is responsible for an operation. A search request is delegated to the Trie, a deadline ordering request to Merge Sort, a range request to Binary Search over a sorted view, and alert operations to the Min-Heap.

F.1 Central Coordination

JobTracker acts as the application-facing abstraction over the individual structures. A caller can add a job without manually inserting it into a set, Trie and heap in a particular order. The tracker performs those coordinated operations and keeps the indexes synchronized.

F.2 Add-Job Flow

validate Job → compute dedup_key → check hash set → store by ID → index searchable text → push deadline
→ return success

F.3 Search Flow

query → normalize prefix → Trie lookup → obtain job IDs → resolve IDs through tracker storage → return Job records

F.4 Alert Flow

requested horizon → inspect heap root → compare deadline → collect due jobs → return preview OR pop for delivery

F.5 Separation of Concerns

The separation between data structures and JobTracker is one of the most important design decisions in the project. It allows the web interface to call high-level operations without knowing how the underlying indexes work. This makes the same DSA engine reusable by the CLI, REST API and future interfaces.

Appendix G: FastAPI Endpoint Reference

The REST API forms the bridge between the DSA engine and the web interface. This appendix records the endpoint purposes, expected request style and returned information at a level suitable for project evaluation.

G.1 Endpoint Design

The API is organized around jobs, alerts and statistics. GET operations retrieve information, while POST operations create jobs or request an e-mail alert. The route layer converts HTTP input into calls to the tracker and converts tracker results into JSON-compatible response models.

G.2 Example Request

POST	/api/jobs
Content-Type:	application/json
{	
"job_id":	"example-001",
"title":	"Example Recruitment",
"department":	"Example Department",
"category":	"Central Government",
"location":	"India",
"qualification":	"Graduate",
"post_date":	"2026-08-20",
"deadline":	"2026-09-05",
"vacancies":	100,
"apply_link":	"https://example.gov.in/apply"
}	

G.3 Duplicate Response

When the same deduplication identity is submitted again, the backend reports a conflict rather than silently creating a second record. This behavior is useful to the front end because it can present a clear duplicate rejection message to the user.

G.4 API Documentation

FastAPI automatically provides interactive OpenAPI documentation. During local development this can be opened at the /docs endpoint. The documentation makes it easier to inspect request schemas and test endpoints without a separate API client.

Appendix H: Frontend and User Workflow

The web interface provides the human-facing presentation layer. The main workflow is to open the dashboard, browse or search postings, inspect job details, review deadlines and optionally add a posting or request an e-mail alert. The UI therefore exposes the same logical capabilities that were previously available through the CLI.

H.1 Dashboard Workflow

The dashboard provides a high-level summary of active postings and deadline activity. It is intended to answer the first user question: what is important right now?

H.2 Explore Jobs Workflow

The Explore Jobs screen provides prefix search, category filtering and deadline sorting. The search request is routed to the backend Trie endpoint, while the sorting option requests deadline ordering from the backend.

H.3 Deadline Alerts Workflow

The alerts page allows the user to choose an alert horizon. The backend performs a non-destructive heap preview, which is appropriate for a page that may be refreshed without consuming the alerts.

H.4 Post Job Workflow

The Post Job form collects the structured fields required by the backend schema. Successful insertion updates the tracker, while a duplicate conflict can be displayed to the user as a validation message.

Appendix I: Testing Matrix

The automated tests are organized around the risk points of the system: correctness of individual structures, correct integration through JobTracker, and correct behavior of the HTTP interface. The test design is intended to detect both algorithmic errors and wiring errors.

I.1 Automated Test Matrix

Area	Examples of verified behaviour
Trie	Prefix search, text tokenization, empty prefix
Min-Heap	Empty operations, ordering, sift behavior
Merge Sort	Numbers, custom Job deadline key, edge cases
Binary Search	Lower/upper bounds, deadline ranges
Hashing	Dedup key normalization, duplicate rejection
JobTracker	Search, sorting, range query, alert preview/pop
API	Jobs, search, closing-soon, alerts, create, stats

I.2 Test Quality

The tests include both ordinary cases and edge cases. Empty data structures, single-element inputs, duplicate submissions and repeated alert previews are important because these are common sources of implementation errors.

I.3 Regression Value

Keeping the original DSA tests after adding the web layer provides regression protection. If a future API or UI change accidentally modifies tracker behavior, the underlying unit tests can identify the problem independently of the browser.

Appendix J: Manual Test Scenarios

Manual testing complements automated tests by checking the complete user flow. These scenarios can be repeated during the final demonstration and are written so that an evaluator can understand what behavior is expected from each screen.

J.1 Search Scenario

Enter a prefix such as “bank” on the Explore Jobs page. The expected result is that relevant postings whose indexed fields begin with the prefix are returned. The purpose of this test is to verify the visible behavior of the Trie-backed search path.

J.2 Deadline Scenario

Open Deadline Alerts and change the alert horizon. The expected result is a set of postings whose deadlines fall within the selected horizon, while repeatedly refreshing the page should not consume the previewed heap entries.

J.3 Duplicate Scenario

Submit a posting whose title, department and post date match an existing posting. The expected result is a conflict response and a clear duplicate rejection message rather than a second record.

J.4 Details Scenario

Open a job card and inspect the details view. The application should show the structured fields and provide the stored application link.

Appendix K: Performance and Complexity Discussion

The project is an academic DSA demonstration rather than a production-scale benchmark. Consequently, complexity analysis is more important than claiming absolute millisecond performance. This appendix explains what the asymptotic costs mean for the current architecture and how the costs would behave as the number of postings grows.

K.1 Asymptotic View

For an academic DSA project, asymptotic complexity explains how the design behaves as n grows. The Trie makes prefix traversal depend on query length, the heap keeps priority operations logarithmic, Merge Sort provides $O(n \log n)$ ordering, Binary Search provides logarithmic boundary lookup on sorted data, and hashing provides average constant-time membership.

K.2 Current Dataset

The current dataset is intentionally small, so wall-clock measurements are not a meaningful demonstration of large-scale scalability. A few dozen records can be processed quickly by almost any reasonable implementation. The stronger evaluation point is that the selected structures have well-defined behavior and are integrated correctly.

K.3 Future Measurement

A future performance study could generate datasets with 10^2 , 10^3 , 10^4 and 10^5 postings, then measure search, insertion, sorting, range queries and alert operations. Such a benchmark should report median or average execution time over repeated runs rather than relying on one measurement.

Appendix L: Security and Reliability Considerations

The current application is a local academic system, but several design decisions already support safer future use. Credentials for e-mail are read from environment variables, request bodies are validated by Pydantic schemas, and duplicate detection prevents accidental repeated records.

L.1 Credential Handling

The e-mail sender and application password are obtained through environment variables. This avoids putting a real credential in source code. The project should continue to exclude environment files and secrets from version control.

L.2 Input Validation

Pydantic schemas provide a validation boundary between HTTP input and the application layer. This helps prevent malformed data from reaching JobTracker and gives the client structured validation errors.

L.3 Deployment Considerations

The current development server and permissive CORS settings are suitable for local evaluation but should not be treated as a production security configuration. A public deployment would require restricted origins, HTTPS, authentication where necessary, secret management and rate limiting.

Appendix M: Feasibility and Deployment Roadmap

The application can be extended without replacing its DSA engine. Persistence, scheduled jobs, live ingestion and deployment can be introduced as surrounding infrastructure. The existing separation between the tracker and presentation layer makes this evolution practical.

M.1 Technical Feasibility

The project uses widely available Python and JavaScript tooling and does not require specialized hardware. The separation between the DSA core, FastAPI layer and React interface allows individual layers to be modified without rewriting the entire system.

M.2 Economic Feasibility

The current project can be developed and demonstrated without paid software. Python, the custom DSA implementations, FastAPI, React and Vite are available through free/open-source tooling. Future hosting or external service costs would depend on the chosen deployment provider and usage.

M.3 Operational Feasibility

The web interface reduces the command-line knowledge required by an end user. The original CLI remains useful for demonstrations and debugging. The application can therefore serve both a technical evaluator and a non-technical user interface.

M.4 Roadmap

Phase	Enhancement	Effect on DSA Core
1	Live data ingestion	Feeds existing Job objects
2	Persistence	Rebuilds indexes from stored records
3	Scheduling	Automates existing scraper/alert operations
4	Deployment	Makes existing web stack remotely accessible
5	Advanced DSA	Adds graph/search enhancements if required

Appendix N: User Guide

This appendix provides a concise operational guide for running the CLI and the full-stack development version. It is intended for an evaluator who wants to reproduce the demonstration locally.

N.1 CLI Demonstration

```
cd govjob_tracker
python main.py --demo
```

The interactive CLI can also be started without `--demo`. The menu presents search, sorting, closing-soon, alerts, e-mail and statistics operations.

N.2 Backend

```
cd govjob_tracker
python -m uvicorn backend.main_api:app --host 127.0.0.1 --port 8000
```

The API is then available locally at port 8000, with interactive documentation at `/docs`.

N.3 Frontend

```
cd govjob_tracker/frontend
npm run dev
```

The Vite development server provides the browser interface. The exact local port may change if the default port is already in use.

N.4 Demonstration Order

- Open the dashboard and show API connectivity.
- Search for a prefix such as “bank”.
- Sort jobs by deadline.
- Open Deadline Alerts and change the horizon.
- Open a job's details.
- Submit a new job and demonstrate duplicate rejection.
- Open the Architecture/DSA explanation.

Appendix O: Academic Learning Outcomes

The project connects classroom DSA concepts with a complete software workflow. The implementation requires translating abstract complexity arguments into interfaces, tests, API endpoints and user-visible behavior.

O.1 DSA Learning

The project converts abstract DSA concepts into concrete operations. The student must reason about invariants, boundary cases and complexity while also connecting those algorithms to application requirements.

O.2 Software Engineering Learning

The project demonstrates modularity, interfaces between layers, API contracts, automated testing and environment-based configuration. These practices complement algorithmic knowledge and show how DSA is used inside a complete application.

O.3 Problem-Solving Learning

A major learning outcome is recognizing that different operations need different structures. A single generic list would be simple but would make prefix search, urgency retrieval and duplicate detection rely on repeated scans. The final design instead uses specialized structures according to the operation.

O.4 Evolution of the Project

Moving from CLI to web also demonstrates abstraction. Because JobTracker provides high-level operations, the presentation layer can change without rewriting the core algorithms. This is a practical example of designing for change.

Appendix P: Future Enhancement Design Notes

The future roadmap is intentionally incremental. Each enhancement is designed to preserve the existing JobTracker abstraction while replacing or extending the surrounding data, storage, notification and presentation layers.

P.1 Persistent Storage

SQLite would allow job records to survive restarts. On application startup, records could be loaded and the existing Trie, Min-Heap and hash indexes rebuilt. The DSA structures would remain responsible for in-memory operations while the database would provide durability.

P.2 Live Sources

A future ingestion layer could parse selected recruitment sources and normalize each record into the existing Job model. The project should respect the terms, access policies and technical constraints of each source rather than assuming that every website can be scraped.

P.3 Personalization

User profiles could store preferred categories, locations, qualifications and alert horizons. Search results and notifications could then be filtered for each user while the existing DSA engine continues to handle the underlying job collection.

P.4 Advanced DSA

A future related-jobs feature could model relationships between postings using a graph, with BFS or DFS used to traverse related categories or skills. Such an addition should be introduced only when the feature has a clear functional purpose.